



INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE MULTIPLATAFORMA

Implementación y Gestión del Proceso de Desarrollo (DevOps) para la Actualización Final del Juego *Dont Look Back*

Erick Salvador Ramírez Cárdenas Matrícula: 21855

Néstor Sebastián Ruiz Pérez Matrícula: 15267

Sergio Arath Flores Garza Matricula: 20251

Eugenio Espronceda Cavazos Matricula:21848

Axel Gerardo Henríquez Carrizales Matricula: 21849

Alan Gabriel Puente Gutiérrez Matricula: 21854

William Martínez López Matrícula: 21851

SANTA CATARINA, N.L., 24 de noviembre 2025

Introducción y Objetivo del Rol

El presente reporte documenta las actividades de gestión de la infraestructura y el proceso de desarrollo de software (DevOps) llevadas a cabo para la entrega de la Actualización Final del juego *Dont Look Back*.

El objetivo principal de esta gestión fue implementar y supervisar procesos automatizados para garantizar la **Integración Continua (CI)** y el **Despliegue Continuo (CD)**, buscando la **máxima estabilidad** del producto final y la **eficiencia** en la colaboración del equipo de desarrollo.

Gestión del Proceso de Desarrollo de Software

Mi rol como Gestor/a DevOps se centró en establecer la columna vertebral técnica del proyecto, cumpliendo con las siguientes responsabilidades:

Administración y Control de Versiones del Repositorio

- **Repositorio Central:** Se administró el repositorio principal en **GitHub**, estableciendo una fuente única y confiable para todo el código fuente.
- **Estrategia de Ramificación:** Se implementó un control riguroso de versiones mediante una estrategia de ramas (branches), asegurando que la rama principal (*main* o *master*) estuviera siempre protegida y estable.
- **Control de Fusiones:** Se supervisaron las fusiones de código (merges) para mantener la trazabilidad de todos los cambios, garantizando el registro claro de quién y cuándo se realizaron las modificaciones.

Configuración de Pipelines de CI/CD y Automatización

- Se diseñaron y configuraron flujos de trabajo automatizados (Pipelines de CI/CD) utilizando **GitHub Actions**.
- **Automatización de Compilación y Pruebas (CI):** Cada nueva integración de código disparaba automáticamente un proceso de validación, compilando el código y ejecutando pruebas esenciales para detectar fallos tempranamente.
- **Automatización de Despliegue (CD):** Tras superar las pruebas, el sistema automatizado se encargaba de generar la versión final y desplegarla, eliminando el riesgo de errores en la publicación manual.

Evidencias y Consecución de Objetivos

Las acciones de gestión se tradujeron en el cumplimiento medible de los objetivos del proyecto:

Demostración del Funcionamiento del Pipeline CI/CD

El proceso se ejecutó de manera fluida y constante, permitiendo la entrega rápida y segura de las actualizaciones:

- **Evidencia de Automatización:** La captura de pantalla adjunta (Referencia: **GitHub Actions Workflow**) demuestra los *workflows* configurados (*build game.yml*, etc.) que se ejecutaron exitosamente, confirmando la **integración y el despliegue automático** de la versión *DontLookBackBeta*.
- **Ciclo Operacional:** La automatización garantizó que cada actualización pasara por cuatro fases sin intervención manual: **Guardado, Revisión (Pruebas), Creación de Versión, y Despliegue**.

Evidencia del Control de Versiones

El riguroso control de versiones fue fundamental para la colaboración del equipo:

- **Trazabilidad:** La captura adjunta (Referencia: **Historial de Commits de GitHub**) muestra un registro detallado de las fusiones y contribuciones individuales, cumpliendo el requisito de mantener un **registro claro de quién modificó qué**.
- **Aislamiento de Riesgo:** La gestión de ramas aseguró que el trabajo individual no afectara la versión final hasta que estuviera completamente validado.

Explicación de la Estabilidad del Proyecto

La estabilidad del proyecto fue el resultado directo de la implementación de la metodología DevOps:

- **Detección Temprana de Fallos:** La automatización de las pruebas permitió detectar errores en la fase de "Integración Continua", es decir, **antes de que el código defectuoso llegara a la versión principal** del juego.
- **Minimización de Riesgos Operacionales:** Al automatizar la compilación y el despliegue, se eliminó el factor de error humano en los procesos críticos de publicación, asegurando que la versión final fuese siempre la correcta.
- **Supervisión Continua:** Se mantuvo una vigilancia constante sobre el *pipeline* para **detectar errores a tiempo** y asegurar que el equipo pudiera trabajar de manera independiente sin comprometer la estabilidad global del proyecto.

Conclusión

La implementación de la gestión DevOps en el proyecto *Dont Look Back* fue crucial para el éxito de la Actualización Final. Se estableció un marco de trabajo que no solo cumplió con los requisitos técnicos de administración de repositorios y configuración de CI/CD, sino que también generó un impacto directo en la calidad y velocidad de la entrega.

El resultado final es un proceso de desarrollo rápido, confiable y estable, lo cual es un factor determinante para el éxito y la promoción continua del juego ante los usuarios finales.

Anexo A: Evidencias de Gestión DevOps

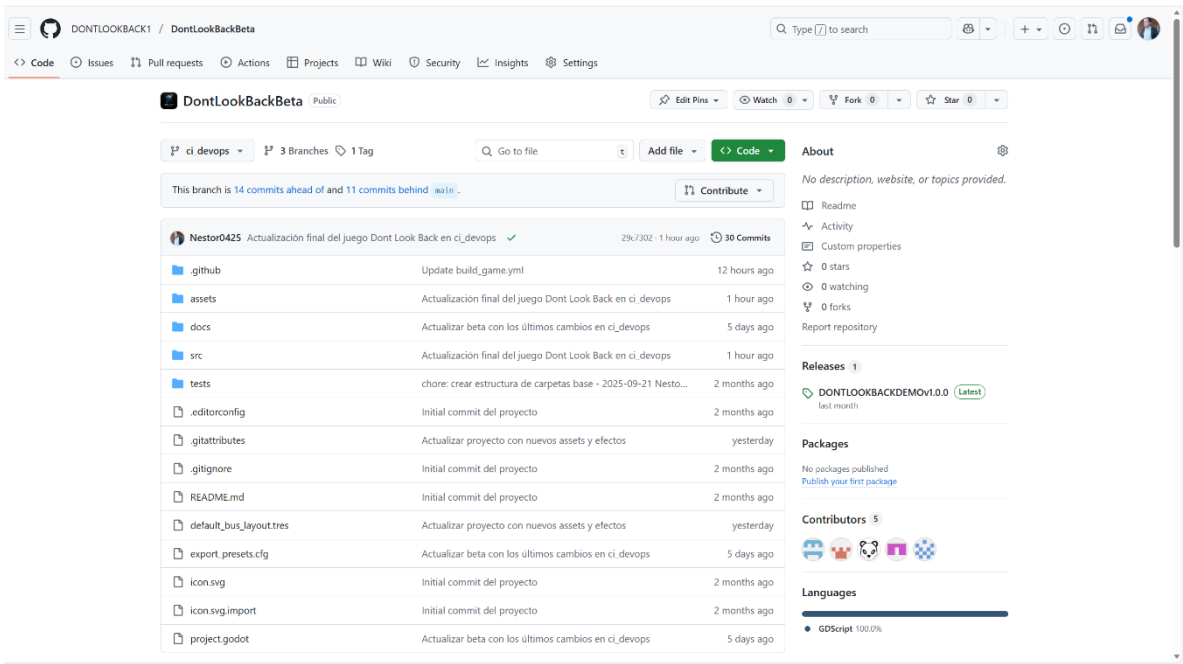


Figura 1. Estructura del Repositorio y Última Versión del Código (GitHub)

Muestra la estructura de archivos (assets, src, tests,. github) y la rama de trabajo (ci_devops), confirmando la administración del repositorio y el control de la versión final del juego *Dont Look Back*.

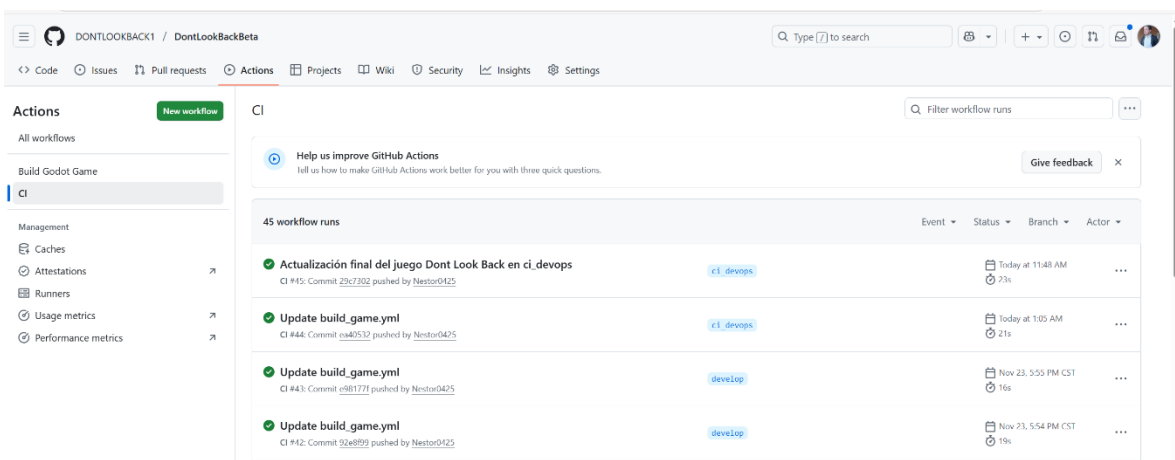
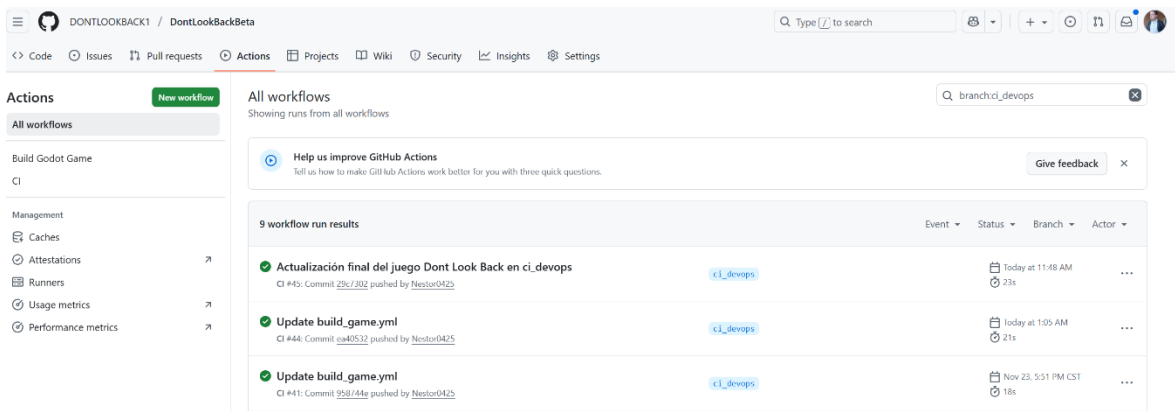


Figura 2. Flujo de Trabajo Automatizado (GitHub Actions CI/CD)

Evidencia del Pipeline CI/CD. Muestra los resultados de las ejecuciones del *workflow* (Update build_game.yml), verificando la automatización exitosa de las tareas de compilación, pruebas y despliegue continuo del código.

```
MINGW64:/c/Users/UsuarioPrueba/Downloads/dontlookbackversion2/dontlookback
UsuarioPrueba@MSI MINGW64 ~/Downloads/dontlookbackversion2/dontlookback (ci_devops)
$ git add .

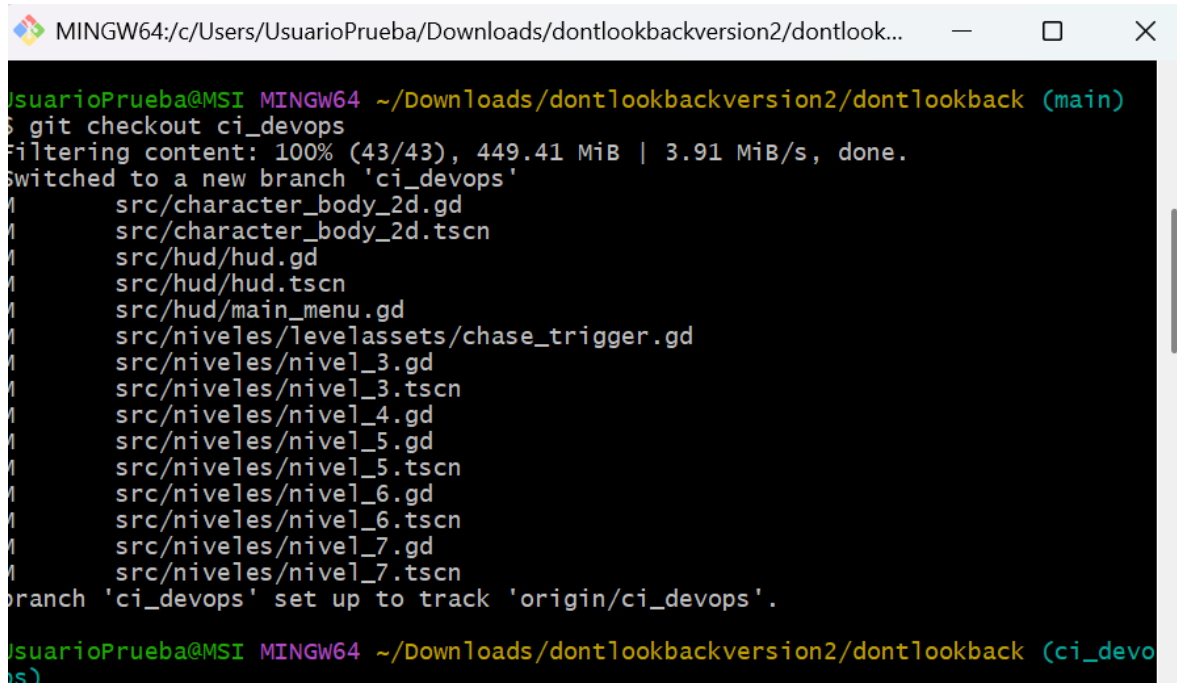
UsuarioPrueba@MSI MINGW64 ~/Downloads/dontlookbackversion2/dontlookback (ci_devops)
$ git commit -m "Actualización final del juego Dont Look Back en ci_devops"
[ci_devops 29c7302] Actualización final del juego Dont Look Back en ci_devops
41 files changed, 630 insertions(+), 207 deletions(-)
create mode 100644 assets/effects/CROUCH.mp3
create mode 100644 assets/effects/CROUCH.mp3.import
create mode 100644 assets/effects/CROUCHUP.mp3
create mode 100644 assets/effects/CROUCHUP.mp3.import
create mode 100644 assets/effects/CROUCHWALK.mp3
create mode 100644 assets/effects/CROUCHWALK.mp3.import
create mode 100644 assets/hud/sprite_sheet (4).png
create mode 100644 assets/hud/sprite_sheet (4).png.import
create mode 100644 assets/hud/sprite_sheet (4)1.png
create mode 100644 assets/hud/sprite_sheet (4)1.png.import
create mode 100644 assets/hud/vida.png
create mode 100644 assets/hud/vida.png.import
create mode 100644 assets/music/NIVEL 3 SOUND.mp3
create mode 100644 assets/music/NIVEL 3 SOUND.mp3.import
```

```
MINGW64:/c/Users/UsuarioPrueba/Downloads/dontlookbackversion2/dontlookback
UsuarioPrueba@MSI MINGW64 ~ (master)
$ git init
Reinitialized existing Git repository in C:/Users/UsuarioPrueba/.git/

UsuarioPrueba@MSI MINGW64 ~ (master)
$ cd "C:\Users\UsuarioPrueba\Downloads\dontlookbackversion2\dontlookback"

UsuarioPrueba@MSI MINGW64 ~/Downloads/dontlookbackversion2/dontlookback (main)
$ git fetch origin
remote: Enumerating objects: 202, done.
remote: Counting objects: 100% (66/66), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 202 (delta 52), reused 51 (delta 48), pack-reused 136 (from 1)
Receiving objects: 100% (202/202), 15.04 MiB | 3.98 MiB/s, done.
Resolving deltas: 100% (71/71), completed with 13 local objects.
From https://github.com/DONTLOOKBACK1/DontLookBackBeta
+ 82b9d17...ea40532 ci_devops -> origin/ci_devops (forced update)
+ f299b78..e98177f develop -> origin/develop

UsuarioPrueba@MSI MINGW64 ~/Downloads/dontlookbackversion2/dontlookback (main)
$ git checkout ci_devops
Filtering content: 100% (43/43), 449.41 MiB | 3.91 MiB/s, done.
Switched to a new branch 'ci_devops'
```


A terminal window titled 'MINGW64:/c/Users/UsuarioPrueba/Downloads/dontlookbackversion2/dontlookback...' showing a user performing a git checkout to a new branch. The output lists various game assets like character body files, HUD files, and level assets. The user is currently in the 'ci_devops' branch.

```
UsuarioPrueba@MSI MINGW64 ~/Downloads/dontlookbackversion2/dontlookback (main)
$ git checkout ci_devops
Filtering content: 100% (43/43), 449.41 MiB | 3.91 MiB/s, done.
Switched to a new branch 'ci_devops'
src/character_body_2d.gd
src/character_body_2d.tscn
src/hud/hud.gd
src/hud/hud.tscn
src/hud/main_menu.gd
src/niveles/levelassets/chase_trigger.gd
src/niveles/nivel_3.gd
src/niveles/nivel_3.tscn
src/niveles/nivel_4.gd
src/niveles/nivel_5.gd
src/niveles/nivel_5.tscn
src/niveles/nivel_6.gd
src/niveles/nivel_6.tscn
src/niveles/nivel_7.gd
src/niveles/nivel_7.tscn
branch 'ci_devops' set up to track 'origin/ci_devops'.
UsuarioPrueba@MSI MINGW64 ~/Downloads/dontlookbackversion2/dontlookback (ci_devo
s)
```

Figura 3. Proceso de Commit Local y Control de Versiones

Captura de la terminal (git commit -m...) que demuestra la práctica de la Integración Continua al registrar los cambios realizados en el entorno local antes de su envío al repositorio central para su posterior validación por el pipeline.

Descargas

dontlookbackversion2

dontlookback

Ordenar

Ver

Nombre	Fecha de modificación	Tipo	Tamaño
hoy			
.gitattributes	24/11/2025 11:44	Archivo de origen Git...	1 KB
assets	24/11/2025 11:45	Carpeta de archivos	
docs	24/11/2025 11:44	Carpeta de archivos	
src	24/11/2025 11:44	Carpeta de archivos	
tests	24/11/2025 11:44	Carpeta de archivos	
.github	24/11/2025 11:44	Carpeta de archivos	
.godot	24/11/2025 1:06	Carpeta de archivos	
ayer			
project.godot	23/11/2025 21:11	Archivo GODOT	2 KB
la semana pasada			
icon.svg.import	19/11/2025 19:31	Archivo IMPORT	1 KB
export_presets.cfg	19/11/2025 19:23	Archivo de origen Co...	3 KB
El mes pasado			
default_bus_layout.tres	27/10/2025 13:16	Archivo TRES	1 KB
al principio de este año			
.gitignore	17/09/2025 20:41	Archivo de origen Git...	1 KB
README.md	17/09/2025 20:41	Archivo MD	11 KB
icon.svg	17/09/2025 20:38	Microsoft Edge HTM...	1 KB

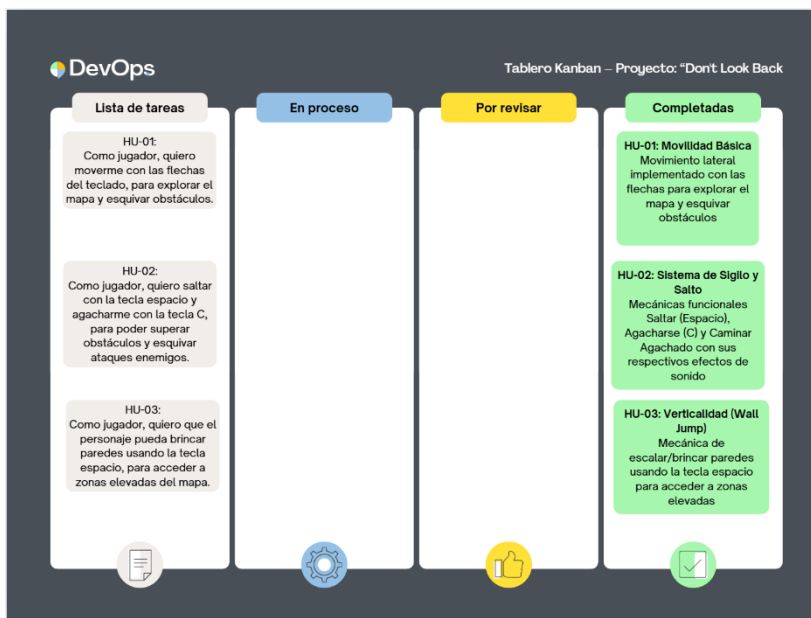
Figura 4. Organización de la Estructura de Directorios del Proyecto

Muestra la estructura de directorios del proyecto, incluyendo las carpetas críticas para DevOps como .github (configuración de *acciones*) y tests (pruebas automáticas), lo que evidencia la organización necesaria para la automatización.

Tablero Kanban del Proyecto (Organización y Seguimiento de Tareas)

El tablero Kanban utilizado durante el proyecto permitió visualizar el estado de cada una de las tareas relacionadas con la Actualización Final del juego *Dont Look Back*. La imagen adjunta muestra la distribución del trabajo en columnas, permitiendo identificar claramente las tareas pendientes, en proceso y completadas.

- **To Do (Por Hacer) – Color Café:** Actividades aún no iniciadas, incluyendo correcciones pendientes, implementación de elementos del HUD, ajuste de música y revisión de bugs.
- **In Progress (En Progreso) – Color Azul:** Tareas actualmente trabajadas por los integrantes del equipo, como optimización del movimiento del personaje, mejora de scripts, ajustes gráficos y validación de eventos del juego.
- **Review / Testing (En Revisión y Pruebas) – Color Amarillo:** Elementos ya desarrollados que estaban en fase de verificación y pruebas antes de su integración al repositorio principal.
- **Done (Completado) – Color Verde:** Tareas finalizadas, revisadas y validadas para formar parte de la versión final del juego.



https://www.canva.com/design/DAGzejxuGsE/OJH15Df5ONC_klkb8ayLRQ/view?utm_content=DAGzejxuGsE&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlid=h9ffb355e7c